

CME 295: Transformers & Large Language Models

Lecture 1 – Study Notes

Based on Lecture 1 by Afshine Amidi & Shervine Amidi

Contents

1	Core Course Abbreviations	2
2	NLP Overview	2
2.1	Learning path	2
2.2	Three broad NLP task types	2
2.3	Representative tasks and metrics	3
2.4	High-level NLP timeline	4
3	Tokenization	5
4	Token / Word Representation	6
4.1	From one-hot vectors to learned embeddings	6
4.2	Word2vec	6
5	RNNs and LSTM	7
5.1	Recurrent Neural Networks	7
5.2	Long Short-Term Memory	8
5.3	Why attention emerged	8
6	Self-Attention and the Transformer	9
6.1	Query, Key, and Value	9
7	Transformer Architecture	10
7.1	Input and positional encoding	11
7.2	Encoder	11
7.3	Decoder	12
7.4	Multi-head attention	13
7.5	Label smoothing	14
8	End-to-End Transformer Example	14

1 Core Course Abbreviations

The lecture introduces many abbreviations. A useful organization is: **Transformer-based models**, **training strategies**, **datasets**, **architectures/techniques**, **NLP tasks**, and **evaluation metrics**. The lecture itself groups the abbreviations this way.

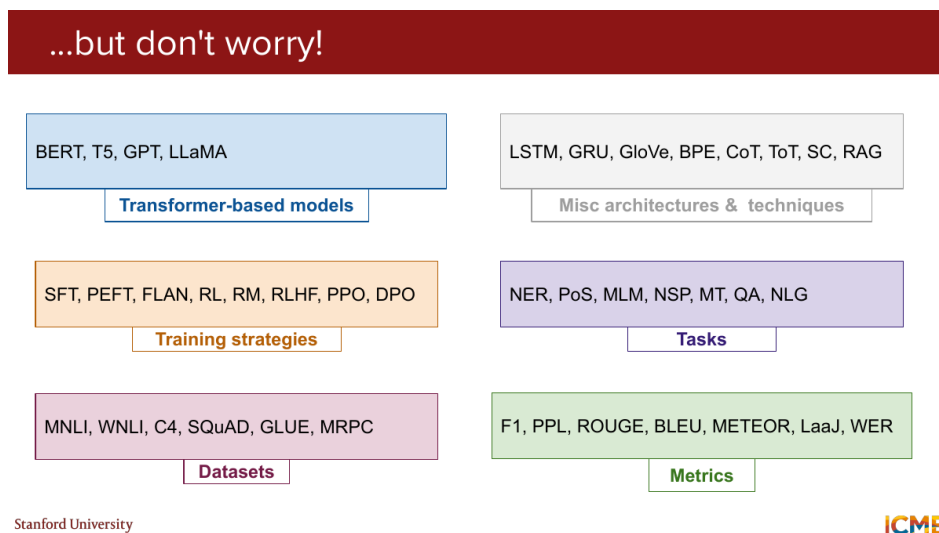


Figure 1: Abbreviations grouped by category. (CME 295 Lecture 1, slide 15).

- **Transformer-based models:** BERT, T5, GPT, LLaMA.
- **Training strategies:** SFT, PEFT, FLAN, RL, RM, RLHF, PPO, DPO.
- **Datasets:** MNLI, WNLI, C4, SQuAD, GLUE, MRPC.
- **Architectures/techniques:** LSTM, GRU, GloVe, BPE, CoT, ToT, SC, RAG.
- **Tasks:** NER, PoS, MLM, NSP, MT, QA, NLG.
- **Metrics:** F1, PPL, ROUGE, BLEU, METEOR, LaaJ, WER.

2 NLP Overview

2.1 Learning path

Tokenization → word representation → RNNs → self-attention → Transformer architecture → end-to-end example.

2.2 Three broad NLP task types

The lecture organizes NLP tasks into **classification**, “multi”-classification, and **generation**. Classification maps text to one label; multi-classification assigns labels to components/tokens; generation produces output text.

NLP tasks overview

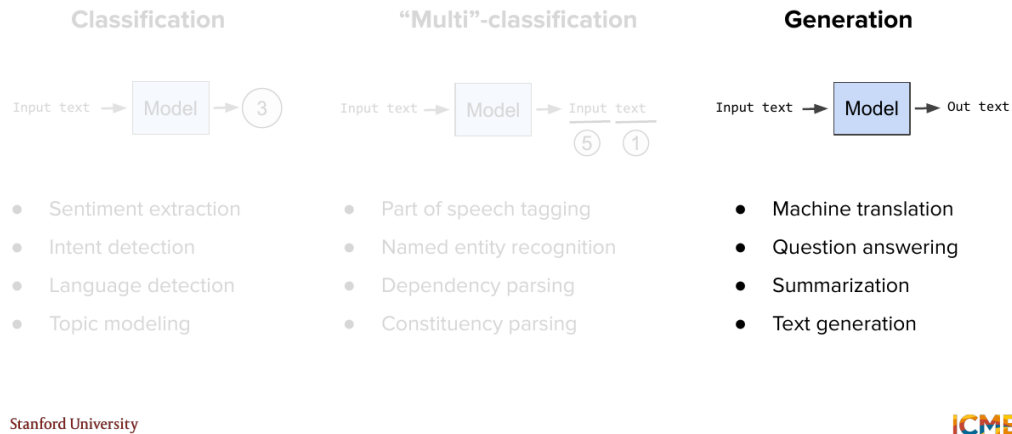


Figure 2: Classification, multi-classification, and generation tasks. (CME 295 Lecture 1, slide 19).

2.3 Representative tasks and metrics

Sentiment extraction. Example: “This teddy bear is SO CUTE!” → positive sentiment. Example datasets include Amazon reviews, IMDB critiques, and Twitter. Metrics include accuracy, precision, recall, and F1.

NLP task: Sentiment Extraction

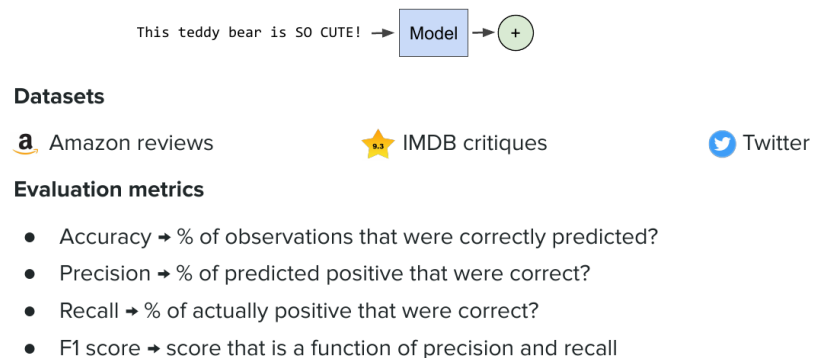


Figure 3: Sentiment extraction datasets and evaluation metrics. (CME 295 Lecture 1, slide 21).

Named Entity Recognition (NER). The model identifies and labels entities in text. The lecture cites annotated Reuters newspaper data (CoNLL-2003, CoNLL++), with accuracy, precision, recall, and F1 evaluated at token level and per entity type.

NLP task: Named Entity Recognition



Datasets

 Annotated Reuters newspaper (CoNLL-2003, CoNLL++)

Evaluation metrics

- Accuracy
- Precision at a token level, per entity type
- Recall
- F1 score

Stanford University

ICME

Figure 4: Named Entity Recognition example and metrics. (CME 295 Lecture 1, slide 23).

Machine Translation (MT). Example: “A cute teddy bear is reading” → “Un ours en peluche mignon lit.” The lecture cites WMT’14 English–French and English–German. BLEU is presented as similar to precision, ROUGE as similar to recall, and perplexity as how “surprised” a model is to see words together.

NLP task: Machine Translation



Datasets

 WMT’14 English-French

 WMT’14 English-German

Evaluation metrics

- BLEU → quality of text translated, similar to “precision”
- ROUGE → quality of text generated, similar to “recall”
- Perplexity → quantifies how ‘surprised’ the model is to see some words together

Stanford University Formulas can be found on this website: <http://www.statmt.org/book/slides/08-evaluation.pdf>

ICME

Figure 5: Machine translation datasets and evaluation metrics. (CME 295 Lecture 1, slide 25).

2.4 High-level NLP timeline

RNNs (1980s) → LSTM (1997) → Word2vec (2013) → Transformers (2017) → Large Language Models (2020s). The lecture contrasts theoretical foundations with the later combination of large datasets, growing compute, and rapid iteration.

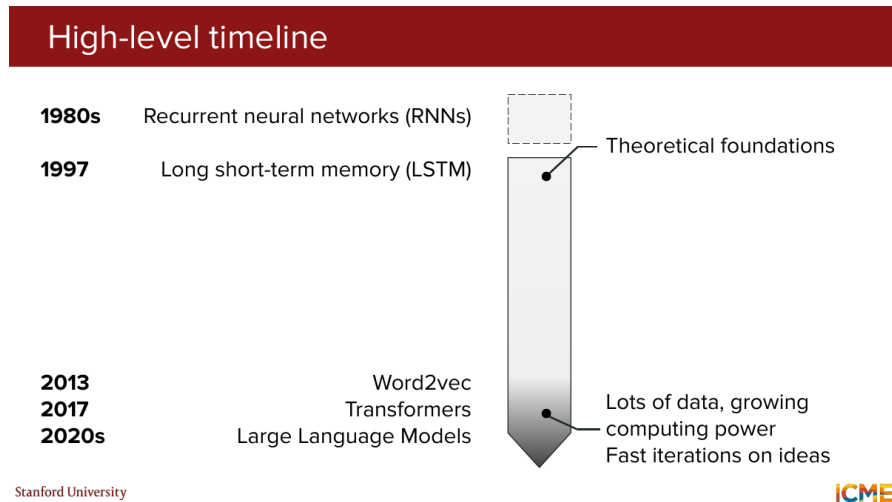


Figure 6: High-level timeline from RNNs to LLMs. (CME 295 Lecture 1, slide 26).

3 Tokenization

Tokenization splits text into units that the model can process. The lecture compares word-level, subword-level (e.g., WordPiece and BPE), and character-level approaches.

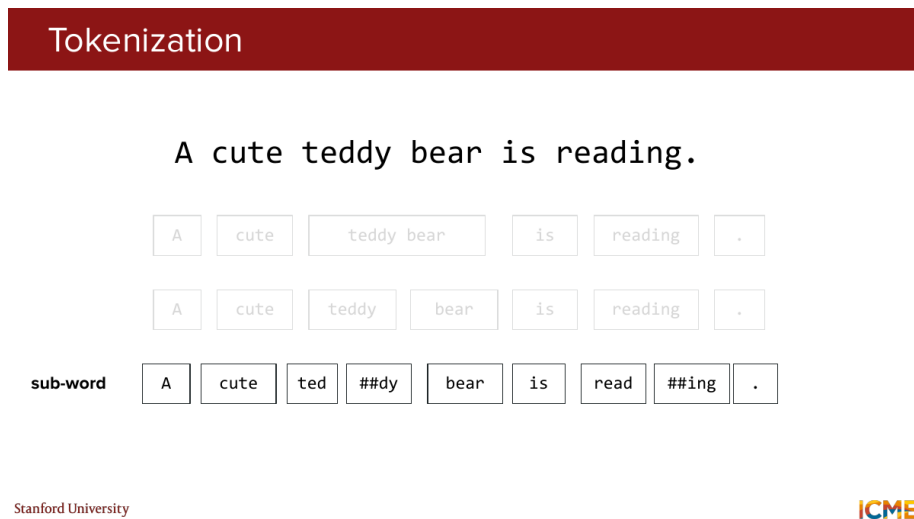


Figure 7: Subword tokenization example: **teddy** and **reading** are decomposed into subword units. (CME 295 Lecture 1, slide 31).

Method	Pros	Cons
Word-level	Simple; interpretable	Risk of OOV; does not leverage root knowledge
Subword-level	Leverages common prefixes/suffixes; learned from data	OOV risk remains, but lower than word-level
Character-level	Small chance of OOV; robust to casing and misspellings	Slower computation; embeddings are less interpretable

Tokenization summary

Method	Pros	Cons
Word-level	- Simple - Interpretable	- Risk of OOV - Does not leverage knowledge of root
Subword-level e.g. WordPiece, BPE	- Leverages common prefixes and suffixes - Learned from the data	Risk of OOV, though less than word-level
Character-level	- Small chance of OOV - RoBUsT to CASinG and MIspeLInGs	- Makes computations slower - Embeddings not interpretable

Stanford University

ICME

Figure 8: Tokenization summary from the lecture. (CME 295 Lecture 1, slide 33).

4 Token / Word Representation

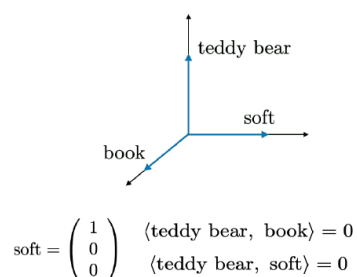
4.1 From one-hot vectors to learned embeddings

One-hot encoding is sparse and does not directly encode semantic similarity. Learned embeddings map tokens to dense vectors so semantically related concepts can occupy nearby directions/regions in representation space.

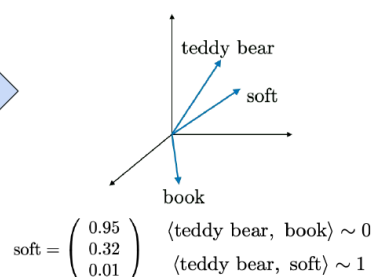
Token representations

Motivation

Naive (one-hot) encoding



Learned embedding



Stanford University

Figure adapted from "VIP cheatsheets for Stanford's CS 230", Amidi, stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

ICME

Figure 9: Motivation for moving from one-hot encoding to learned embeddings. (CME 295 Lecture 1, slide 36).

4.2 Word2vec

Word2vec learns an embedding layer through a proxy task over a large text corpus. The lecture highlights **CBOW**, which predicts a center word from context, and **Skip-gram**, which predicts context from a

center word.

Word2vec

Overview

- Neural network with a **proxy task** over billions of words worth of text
- Learns an embedding layer

Proxy tasks

- CBOW (continuous bag of words)

...A cute teddy bear is reading...

- Skip-gram

...A cute teddy bear is reading...

Stanford University Word2vec paper "Efficient Estimation of Word Representations in Vector Space", Mikolov et al., 2013.

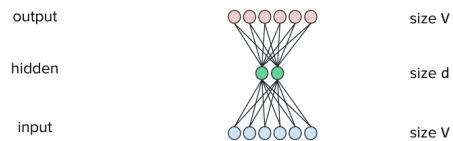
ICME

Figure 10: CBOW and Skip-gram proxy tasks. (CME 295 Lecture 1, slide 37).

The simplified architecture is input size $V \rightarrow$ hidden embedding dimension $d \rightarrow$ output size V , where V is vocabulary size.

Word2vec

Architecture



Stanford University

ICME

Figure 11: Simplified Word2vec architecture. (CME 295 Lecture 1, slide 38).

5 RNNs and LSTM

5.1 Recurrent Neural Networks

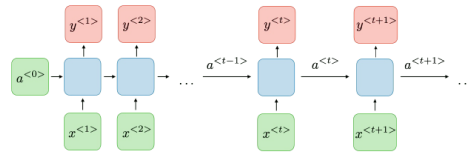
RNN connections form a temporal sequence. Tokens are processed sequentially and the hidden state carries information forward. This lets RNNs represent order, but sequential computation is slow and long-range dependencies are difficult because of gradient issues.

Recurrent Neural Networks (RNNs)

Overview

- First introduced in the 80s
- Class of neural networks where connections form a temporal sequence

General form



Stanford University Figure adapted from "VIP cheatsheets for Stanford's CS 230", Amidi, stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

ICME

Figure 12: General RNN concept introduced in the lecture. (CME 295 Lecture 1, slide 51).

5.2 Long Short-Term Memory

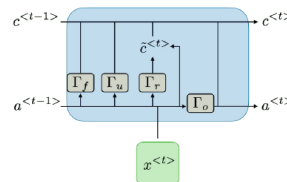
LSTM, introduced in 1997, uses a more structured recurrent unit with gating to preserve or discard information. It improves long-range information flow compared with a basic RNN, while remaining sequential.

Long Short-Term Memory (LSTM)

Overview

- Introduced in "Long short-term memory" (1997)
- Uses a more structured approach in the cell's hidden state

General form



Stanford University Figure adapted from "VIP cheatsheets for Stanford's CS 230", Amidi, stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

ICME

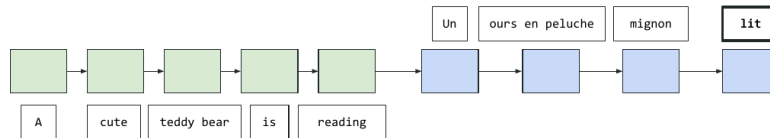
Figure 13: LSTM overview. (CME 295 Lecture 1, slide 63).

5.3 Why attention emerged

The lecture motivates attention through machine translation: long-term dependencies make it difficult for a fixed sequential representation to preserve all relevant source information. Attention lets the decoder focus on the relevant source parts during generation.

History of attention

- Introduced in 2014
- Translation tasks had a real issue with long-term dependencies
- Seq2seq unable to "remember" what input sentence was saying



Stanford University "Neural Machine Translation by Jointly Learning to Align and Translate", Bahdanau et al., 2014.

ICME

Figure 14: Attention addressing a long-term dependency in translation. (CME 295 Lecture 1, slide 68).

6 Self-Attention and the Transformer

The Transformer was introduced in the 2017 paper *Attention Is All You Need*, relies on self-attention, and was presented in the lecture as achieving state-of-the-art machine translation results.

Overview of the Transformer

- Introduced in the 2017 paper "**Attention is All You Need**"
- Relies on the **self-attention** mechanism
- State-of-the-art results on machine translation tasks

Stanford University

ICME

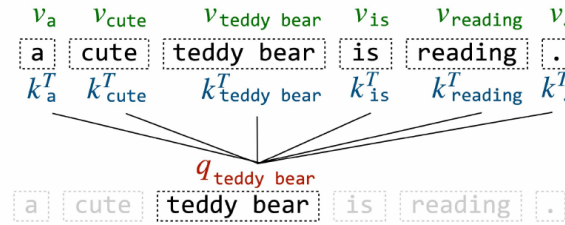
Figure 15: Transformer overview. (CME 295 Lecture 1, slide 70).

6.1 Query, Key, and Value

For each token, learned linear transformations produce **Query** (Q), **Key** (K), and **Value** (V) representations. Queries are compared with keys to determine which values should receive greater attention.

Attention mechanism

Concept of **Q**uery, **K**ey, **V**alue



Stanford University "Super Study Guide: Transformers & Large Language Models", by Amidi, 2024

ICME

Figure 16: Lecture illustration of Query, Key, and Value. (CME 295 Lecture 1, slide 76).

A standard scaled dot-product attention computation is

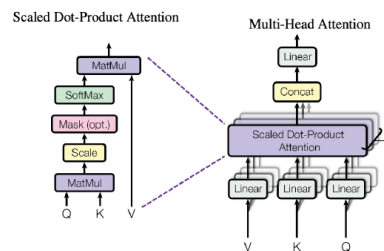
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Conceptually: compute query–key compatibility scores, scale by $\sqrt{d_k}$, normalize with softmax, and use those weights to combine the values.

Attention mechanism

Efficient computations with matrices:

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

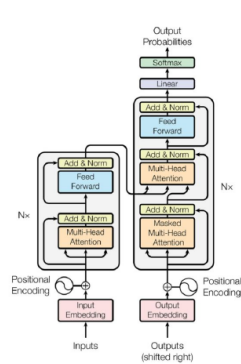
ICME

Figure 17: Matrix-based attention computation. (CME 295 Lecture 1, slide 78).

7 Transformer Architecture

The lecture breaks the architecture into attention layers, feed-forward neural networks (FFNNs), and positional encoding, organized into encoder and decoder stacks.

Transformer architecture



Attention layer (MHA)

- Self-attention (Encoder-Encoder, Decoder-Decoder)
- Encoder-Decoder attention layer

Feed Forward Neural Network (FFNN)

Positional Encoding (PE)

Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

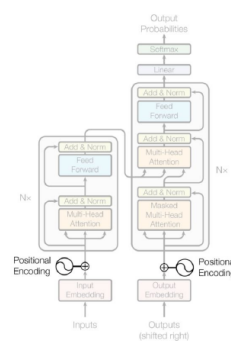
ICME

Figure 18: Overall Transformer architecture and its major modules. (CME 295 Lecture 1, slide 80).

7.1 Input and positional encoding

Text is tokenized and mapped to learned embeddings. Because the Transformer does not obtain order from recurrent computation, positional information is added to the input. The lecture notes that positional encoding may be learned or hard-coded, with the goal of allowing the model to understand relative input position.

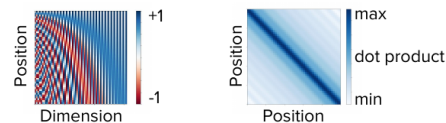
... with a trick!



Positional encoding

Idea:

- Add **position information** to inputs
- Can be either learned or hardcoded



Goal: let model understand relative input position

Stanford University Left figure adapted from "Attention Is All You Need", Vaswani et al., 2017. Right figures adapted from "Transformer Architecture: The Positional Encoding", Kazemnejad, 2019.

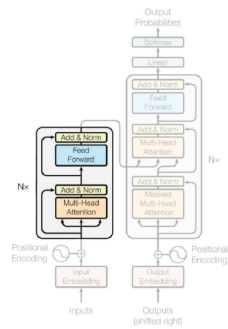
ICME

Figure 19: Positional encoding: adding position information to token embeddings. (CME 295 Lecture 1, slide 82).

7.2 Encoder

Each encoder layer contains encoder-encoder self-attention, a feed-forward neural network, and normalization. Important parameters include the number of stacked layers N , number of heads h , d_{model} , and sub-layer dimensions such as d_{FF} , d_{key} , and d_{value} .

Encoder



Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

ICME

Overview

- Encoder-Encoder attention / self-attention
- Feed Forward Neural Network
- Normalization layer

Parameters

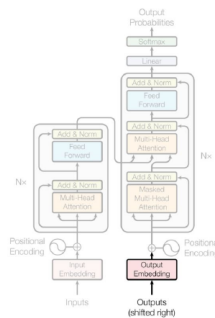
- N: layers stacked
- h: number of attention heads
- d_{FF} , d_{key} , d_{value} : sub-layer dimension
- d_{model} : embedding dimensions

Figure 20: Encoder components and key dimensions. (CME 295 Lecture 1, slide 83).

7.3 Decoder

The output sequence is shifted right and begins with [BOS] during translation. Decoder layers contain decoder self-attention, encoder-decoder attention, feed-forward layers, and normalization. Masking prevents a position from using future output tokens during autoregressive generation.

Output "shifted right"



Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

ICME

Overview

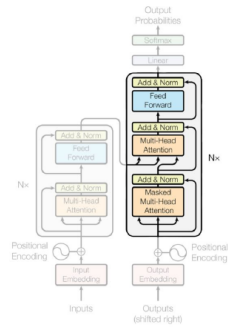
- Learned embeddings for output tokens
- In practice, will start with [BOS] during translation

Parameters

- V: vocabulary size
- d_{model} : embedding dimensions

Figure 21: Target/output tokens shifted right before entering the decoder. (CME 295 Lecture 1, slide 84).

Decoder



Overview

- Decoder-Decoder attention / self-attention
- Encoder-Decoder attention
- Feed Forward Neural Network
- Normalization layer

Parameters

- N: layers stacked
- h: number of attention heads
- d_{FF} , d_{key} , d_{value} : sub-layer dimension
- d_{model} : embedding dimensions

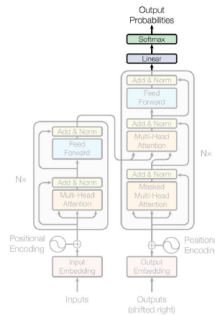
Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

ICME

Figure 22: Decoder components. (CME 295 Lecture 1, slide 85).

The final representation is linearly projected to vocabulary logits and converted to a probability distribution over output-word classes.

Output



Overview

- Linear projection
- Classification problem that outputs probability of belonging to a class, where class = word

Parameters

- V: vocabulary size
- d_{model} : embedding dimensions

Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

ICME

Figure 23: Output projection to vocabulary probabilities. (CME 295 Lecture 1, slide 86).

7.4 Multi-head attention

Instead of running one attention operation, multi-head attention runs multiple attention heads in parallel. This allows different heads to capture different attention features, analogous in the lecture to multiple convolutional filters in computer vision.

Computational tricks

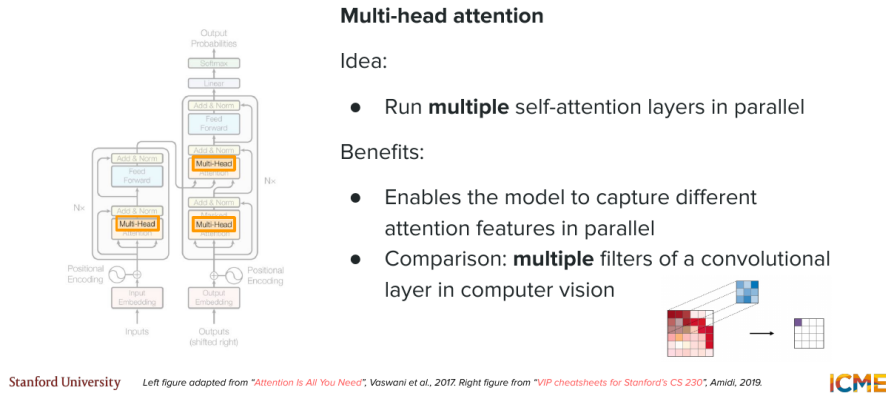


Figure 24: Multi-head attention as parallel attention feature extraction. (CME 295 Lecture 1, slide 87).

7.5 Label smoothing

Label smoothing introduces noise into the true labels to reduce overconfidence and overfitting. A common expression is

$$q'(k | x) = (1 - \epsilon) \delta_{k,y} + \epsilon u(k).$$

The lecture notes improved generalization, accuracy, and BLEU as benefits.

Computational tricks

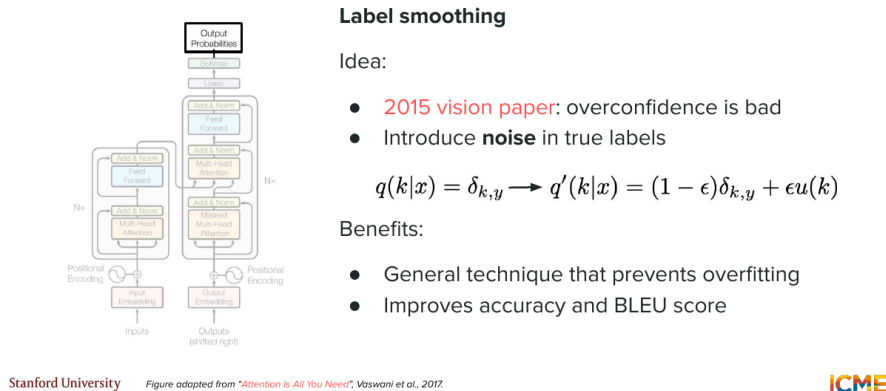


Figure 25: Label smoothing computational trick. (CME 295 Lecture 1, slide 88).

8 End-to-End Transformer Example

The lecture concludes by stitching the pieces together for English-to-French translation. The source sentence is tokenized, embedded, and position-encoded; the encoder builds contextual representations; the decoder starts from [BOS] and generates the target sequence autoregressively, using masked self-attention plus encoder-decoder attention until the sequence is complete.

Stitching all the pieces together with an example



Figure 26: Beginning of the lecture's end-to-end Transformer example. (CME 295 Lecture 1, slide 90).

Conceptual generation loop

1. Tokenize the source sentence and create learned token embeddings.
2. Add positional information and pass the representations through the encoder stack.
3. Start the decoder input with [BOS] and apply masked decoder self-attention.
4. Use encoder-decoder attention so the decoder can condition on relevant source representations.
5. Project the decoder output to vocabulary probabilities and generate the next token.
6. Append the generated token and repeat autoregressively until the end-of-sequence token is produced.

9 Core Takeaways

1. NLP sequence modeling progressed from static word representations to recurrent models and then to attention-based Transformers.
2. Tokenization determines the basic units the model sees; subword methods balance vocabulary coverage and efficiency.
3. Learned embeddings encode useful relationships that one-hot vectors do not.
4. RNN/LSTM models capture sequence order but rely on sequential computation.
5. Self-attention uses Q/K/V representations to dynamically combine information across tokens.
6. Transformer encoder/decoder stacks combine attention, FFNNs, normalization, and positional information.
7. Multi-head attention captures different attention features in parallel, and autoregressive masking prevents access to future target tokens.